Faculty of Engineering, Environment and Computing
6004CMD Mobile App Development

Assignment Brief and Grading Rubric 2024-2025

Module Title Mobile App Development	Ind/Group Individual	Cohort Jan/May 25/26	Module Code 6004CMD
Coursework Title Designing and Building a Mobile App			Hand Out Date 16/01/2026
Lecturer Mr. Bogdan Boldur			Due Date 13/04/2026 @18:00
Estimated Time (hrs): Word Limit*: N/A	Coursework Type: Portfolio		100% of Module Mark

Submission Arrangement Online via Aula: A text file with links (Video Link + Github Repository) uploaded through the assignment link.

File Type:
A PDF file with two links:

- A 8-minute demonstration video uploaded to YouTube
- A Coventry University GitHub Repository link where you hosted the project

Mark and Feedback Method: Rubric marks and comments

Failing to submit the Coventry University GitHub Repository link will result in a zero mark for this assessment.

Module Learning Outcomes Assessed

1. Design and implement a mobile application, showing systematic knowledge of relevant tools, methods and processes.

2. Understand and critically evaluate the different tools and techniques for mobile application development.

3. Scope, design, implement and critically evaluate a basic security policy to keep confidential data safe on a mobile device.

4. Demonstrate an ability to maintain ease of data access/usability across several platforms.

5.Understand,articulate and provide sustained argument on how Cloud Computing can be used to enhance the mobile computing in business.

Assignment Brief

You are required to design and build a .NET Maui app that demonstrates your proficiency in the skills that have been taught during the module.

The application must be developed using the C# programming language and .NET Maui framework.

The app idea can be anything you can think of (e.g., Weather app, Notes app, Recipes app, Fitness apps etc.). However, please carefully read the grading rubrics to make sure when you build your app, you take into consideration all the necessary aspects to get a high grade.

You can use any kind of 3rd party libraries if they are not paid for or would in any shape or form generate the application for you (please consult with the module leader if in doubt).

You are encouraged to use AI coding ***assistants (e.g., GitHub Copilot, Claude Desktop)*** in order to speed up your development.

You may use sample code from the labs, videos, demos or any other online sources referenced throughout this module if you understand why you used it. However, <u>you must annotate the code if you copied bigger chunks and specify the reason for copying it</u>. Copying code from other sources and not properly referencing it will be dealt with ***accordingly***.

Submission file Type: ***<u>PDF</u>***

Submission file requirements:

1. GitHub Repository

    a. As part of this submission, you need to create a repository in the 6004CMD organisation https://github.coventry.ac.uk/6004CMD-25-26 . Your individual repository should be named <u>6004CMD_StudentName</u>

        i. NOTE: Under exceptional circumstances, in case you cannot access the organisation due to unforeseen circumstances, you might be allowed to create a repository under your personal GitHub but you must first speak with me.

    b. The repository **MUST** have a Readme file containing the name of the application and a short background/motivation. This must not be more than 700 words.

2. Video Link:

   a. You are required to create a **video** that clearly showcases the application you have developed. As part of this video, you should show each screen that you can navigate to within the application, any functionality you have on these screens and any external APIs or Cloud providers that you have used. **Guidelines to follow can be found below:**

      i. If you use any external APIs or Cloud providers for authentication services or data persistence, you must make sure that you provide evidence in your submission file on how these providers were used in your project. Use the below guideline:
         - Have evidence of usage of these cloud providers - images with the dashboards showing clear connection with your project or snapshots of code showing clear usage of that cloud provider API in your project

      ii. The video must be no longer than 8 minutes (+/- 10%) and have a quality of at least 720HD. Show your app running on the screen and how to use it.

      The video link must be uploaded on YouTube and be provided in the submission file.

      iii. The video must have VOICE ***OR*** SUBTITLES to avoid confusion.

1. Please notify your registry course support team and module leader for disability support.

2. Any student requiring an extension or deferral should follow the university process as outlined here.

3. The University cannot take responsibility for any coursework lost or corrupted on disks, laptops or personal computer. Students should therefore regularly back-up any work and are advised to save it on the University system.

4. If there are technical or performance issues that prevent students submitting coursework through the online coursework submission system on the day of a coursework deadline, please note there is a grace period. This grace period will normally be 24 hours from the original deadline.

5. You are encouraged to check the originality of your work by using the draft Turnitin links on Aula.

6. Collusion between students (where sections of your work are similar to the work submitted by other students in this or previous module cohorts) is taken extremely seriously and will be reported to the academic conduct panel. This applies to both coursework and exam answers.

7. You must not submit work for assessment that you have already submitted (partially or in full), either for your current course or for another qualification of this university, with the exception of resits, where for the coursework, you may be asked to rework and improve a previous attempt. This requirement will be specifically detailed in your assignment brief or specific course or module information. Where earlier work by you is citable, i.e. it has already been published/submitted, you must reference it clearly. Identical pieces of work submitted concurrently may also be considered to be self- plagiarism.

This document is for Coventry University students for their own use in completing their assessed work for this module and should not be passed to third parties orposted on any website. Any infringements of this rule should be reported to facultyregistry.eec@coventry.ac.uk.

Marking scheme		No attempt (0.00)	Very poor (20.00)	Poor (40.00)	Good (60.00)	Excellent (80.00)	Outstanding (100.00)
UI Usage (20%)		No UI observed. No styling applied.	Very limited usage of UI (very basic UI complexity*).	Limited usage of UI can be observed (basic UI complexity*).	Good usage of UI can be observed (Medium UI complexity*).	Very good usage of UI can be observed (Medium-Complex complexity*).	Outstanding usage of UI can be observed (Comprehensive complexity* and at least an attempt to create a custom control).
Version control (15%)		No link provided or link provided that leads to an empty repository.	Link provided, valid repository (read-me included) Less than 6 commits Less than one week commit history One branch No merges present.	Link provided, valid repository (read-me included) At least 6 commits At least one week commit history One branch No merges present.	Link provided, valid repository (read-me included) At least 10 commits spaced regularly during the development time At least one week commit history Two branches (main – for the release and development) At least one Merge from development back into release.	Link provided, valid repository (read-me included) At least 12 commits spaced regularly during the development time At least Two branches (main and development) At least two weeks commit history Attempted to use feature branches (one) for individual features. One merge from a feature branch into development. One merge from development back into main.	Link provided, valid repository (read-me included) At least 12 commits spaced regularly during the development time At Least Two branches (main and development) At least two weeks commit history Clear usage of feature branches (At least two) for individual features At least two merges from a feature branch into development and from development in main
App architecture (45%)	**MVVM integration (25%)**	No attempt at integration MVVM can be seen. All the business logic is in the code-behind.	Very limited attempt at integration of MVVM There is very limited attempt to use 1 view-model Most of the code still resides in code-behind.	Limited attempt at integration of MVVM At least 2 view-models have been implemented correctly in the app There is still a lot of code behind in the other pages of the app.	Adequate attempt at integration of MVVM At least 4 correctly implemented view models can be observed There is **some** evidence of attempt to use a base view model (like ObservableObject) for adherence to DRY principle and common code used by all viewmodels There is still some code-behind left in some of the pages but overall, not worrying.	Good attempt at integration of MVVM At least 6 correctly implemented view models can be observed There is **some** evidence of attempt to use a base view model (like ObservableObject) for adherence to DRY principle and common code used by all viewmodels There is very little code behind left across the app.	Comprehensive attempt at integration of MVVM. At 8 eight correctly implemented view models can be observed There is **some** evidence of attempt to use a base view model (like ObservableObject) for adherence to DRY principle and common code used by all viewmodels No code-behind can be found in the app.
	Software design patterns (10%)	No software design pattern used.	One software design pattern used or at least attempted**.	One software design pattern correctly used**.	Two software design patterns used**.	Three software design patterns used**.	Four or more software design patterns used**
	Separation of concerns (10%)	No attempt to separate the concerns The whole application is residing under the root project.	Very poor separation of concerns within the app Not all of Models/View-models/Services/Interfaces have their own folders A least 1 of the above categories can be seen in the project structure.	Poor separation of concerns Not all of Models/View-models/Services/Interfaces have their own folders but At least 2 of the above categories can be seen in the project structure.	Good separation of concerns Not all of Models/View-models/Services/Interfaces have their own folders At least 3 of the above categories can be seen in the project structure.	Very Good separation of concerns Not all of Models/View-models/Services/Interfaces have their own folders but At least 4 of the above categories can be seen in the project structure.	Outstanding separation of concerns All of Models/View-models/Services/Interfaces have their own folders respectively There are other folders as well for different functions of the app such as Converters.
Database integration (10%)		No database integration	Very limited database integration. The integration crashes when attempt to see the results of any available query Local integration.	Limited database integration. At two one CRUD operations present Local integration.	Good database integration. At least three CRUD operations present Local integration.	Comprehensive database integration. At least three CRUD operations present Cloud integration	Comprehensive database integration. All CRUD operations can be seen. Cloud integration
Cloud integration (10%)		No cloud integration observed	Very limited cloud integration. Very limited (not working/partially working) attempt at one API integration (authentication-authorisation ***or*** database integration ***or*** public API integration)	Limited cloud integration. Limited attempt(working) to integrate one external API(authentication-authorisation ***or*** database integration ***or*** public API integration) At least 1 endpoint working.	Good cloud integration. One external API integrated (authentication-authorisation ***or*** database integration ***or*** public API integration) At least 2 endpoints available and working	Very good cloud integration. Two external API integrated. One must be authentication/authorisation integration. Second can be any public/database API integration. 3 endpoints available and working	Excellent cloud integration. Two or more external APIs integrated. One must be authentication-authorisation. Second must be database integration. At least 4 endpoints available and working

NOTES:

1. *UI Complexity:

    Very basic UI complexity:

        At least 3 different UI controls are seen across the app.

    Basic UI complexity:

        At least 5 different UI controls are seen across the app.

        Must include at least one type of the following layouts: VerticalStackLayout, HorizontalStackLayout, Grid or FlexLayout.

    Medium UI complexity:

        At least 7 different UI controls are seen across the app.

        Must include the following UI controls: CollectionView

    Medium-Complex UI complexity:

        At least 7 different UI controls are seen across the app.

        Must include one type of the following: Flyouts(with at least 3 working links) or TabBar (with at least 3 tabs).

    Comprehensive UI complexity:

        At least 8 different UI controls are seen across the app.

        At least an attempt to build your own Control.

2. To be awarded Medium level for UI complexity, you must first make sure you have the requirements for the previous levels achieved.

   For example, if you aim for Medium UI complexity, you need to have reached the previous levels by making sure you have at least 7 different UI elements (Which covers Very Basic usage, Basic usage and Medium usage), include either a StackLayout or a Gridlayout and have at least one CollectionView in your app.

3. **Does Not include the fact .NET Maui comes pre-loaded with IoC (Inversion of Control) and DI (Dependency Injection) if I cannot see a clear usage of these patterns.